Administration & Tuning des Bases de données

5

Structures d'optimisation des requêtes

L'index Bitmap

Index Bitmap

- **Un index bitmap** est organisé sous la forme d'un tableau de bits, très pratique pour des opérations booléennes.
  - tableau qui contient autant de colonnes que de valeurs possibles de la clé, et autant de lignes que la relation à indexer.
  - chaque case (x, y) contient un bit qui indique si la ligne x a la valeur de clé y ; la ligne ne comporte que des 0 si la valeur de la clé vaut null

RoID	SSI	IV	SII
113	1	0	0
223	0	1	0
254	1	0	0
300	0	0	0
320	0	0	1

Index bitmap

RoID	Matricule	Nom	specialité
113	1		SSI
223	2		IV
254	3		SSI
300	4		null
320	5		SII

Table « étudiant »

Exemple de requetes

- Nombre d'étudiant de la spécialité SSI
  - **SELECT** count(*) **FROM** etudiant **WHERE** Specialité ='SSI'

RolD	SSI	IV
113	1	0
223	0	1
254	1	0
300	0	0
320	0	0

  - On compte le nombre de 1 dans le tableau SSI

Exemple de requêtes

- Nombre d'etudiant de la specialité SSI
  - **Select** count(*) **FROM** etudiant **WHERE** Specialité ='SSI' **OR** Specialité='SII'

RolD	SSI	IV	Sii
113	1	0	O
223	0	1	0
254	1	0	0
300	0	0	0
320	0	0	1

  - On compte le nombre de 1 dans le tableau SSI et SII

Principe de l'index Bitmap

Soit un attribut A, ayant prenant n valeurs possibles (V1, …, Vn)

Création d'un index bitmap sur l'attribut A:

- On crée n **tableaux** de **n bits**, un pour chaque ***valeur vi***
- Le tableau contient un bit pour chaque tuple t
- Le bit d'un tuple t est à 1 si t.A=vi, à 0 sinon

Création d'un index bitmap

CREATEBITMAP INDEX nom_index **ON** Nom_TABLE.Nom_attribut

Exemple 2

Table « Enseignant »: un index bitmap est définit sur l'attribut grade

ROWID(RID)	ID_ENS	Nom	Grade
00076:000:0123	1	Kadi	Prof
00123:045:8954	2	Melyani	MCA
03458:020:6543	3	Riahi	Prof
12065:213:1003	4	Slimani	MCB
16065:654:1234	5	Fertoul	MCB
23876:020:344	6	Berzeg	MAA

ROWID(RID)	Prof	MCA	MCB	MAA
00076:000:0123	1	0	0	0
00123:045:8954	0	1	0	0
03458:020:6543	1	0	0	0
12065:213:1003	0	0	1	0
16065:654:1234	0	0	1	0
23876:020:344	0	0	0	1

Utilisation de l'index Bitmap

- **Exemple:**
- **SELECT** *
  FROM Enseignant
  WHERE Grade ='Prof'
  - Charger l'index en RAM
  - Comme la restriction est définie sur grade =Prof
  - seules les RowID des TupleS ayant la valeur 1 dans la colonne prof seront récupérés
  - Chargé les enregistrements des ces RowID

ROWID(RID)	Prof	MCA	MCB	MCA
00076:000:0123	1	0	0	0
00123:045:8954	0	1	0	0
03458:020:6543	1	0	0	0
12065:213:1003	0	0	1	0
16065:654:1234	0	0	1	0
23876:020:344	0	1	0	0

Utilisation de l'index Bitmap

- **Exemple:** Select * From Enseignant whre Grade ='MCB' OR grade ='MCA'

ROWID(RID)	Prof	MCA	MCB	MCA
00076:000:0123	1	0	0	0
00123:045:8954	0	1	0	0
03458:020:6543	1	0	0	0
12065:213:1003	0	0	1	0
16065:654:1234	0	0	1	0
23876:020:344	0	1	0	0

MCB		MCA		MCA
0		0		0
0		1		1
0	v	0	=	0
1		0		1
1		0		1
0		0		0

Index BITMAP

- Ce type d'index permet de précalculer une sélection
- Index bitmap permet d'optimiser les opérations des sélections, comparaison
- Les index bitmap est intéressant pour les requêtes de types select count(*)

Index de jointure

- Précalculer une opération de jointure
- Utilisé avec les table liées avec des clés étrangères avec plusieurs table
- Maintient des relations entre une clé étrangère avec les clés primaire

L'index jointure

Index de jointure		
IdligneVente	idLigne client	IdLigneArticle
V1	C1	A1
V2	C2	A2
V3	C3	A1
V4	C1	A1

Table Ventes				
IdLigne (ROWID)	NoClient	NoArticle	dateVente	Montant
V1	1	10	12-03-2015	100
V2	2	20	14-03-2017	200
V3	3	10	12-04-2020	500
V4	1	10	12-03-2023	150

Table Client		
idLigne (ROWID)	NoClient	nomVille
C1	1	Alger
C2	2	Oran
C3	3	Alger
C4	4	Blida

Table Article		
IdLigne (ROWID)	NoArticle	catégorie
A1	10	Santé
A2	20	Santé
A3	40	Santé
A4	50	AcceSport
A5	60	AcceSport

Calculer le montant d'achats des articles accessoires de sport

Index de jointure		
IdligneVente	idLigne client	IdLigneArticle
V1	C1	A1
V2	C2	A2
V3	C3	A1
V4	C1	A1
V5	C1	A5

Table Article		
IdLigne (ROWID)	NoArticle	catégorie
A1	10	Santé
A2	20	Santé
A3	40	Santé
A4	50	AcceSport
A5	60	AcceSport

Table Ventes				
IdLigne (ROWID)	NoClient	NoArticle	dateVente	Montant
V1	1	10	12-03-2015	100
V2	2	20	14-03-2017	200
V3	3	10	12-04-2020	500
V4	1	10	12-03-2023	150
V5	1	60	12-6-2023	400

Charger les enregistrements
Pas de jointure

Calculer le montant d'achats des cleints d'Alger ayant acheté des article de santé

Index de jointure		
IdligneVente	idLigne client	IdLigneArticle
V1	C1	A1
V2	C2	A2
V3	C3	A1
V4	C1	A1
V5	C1	A5

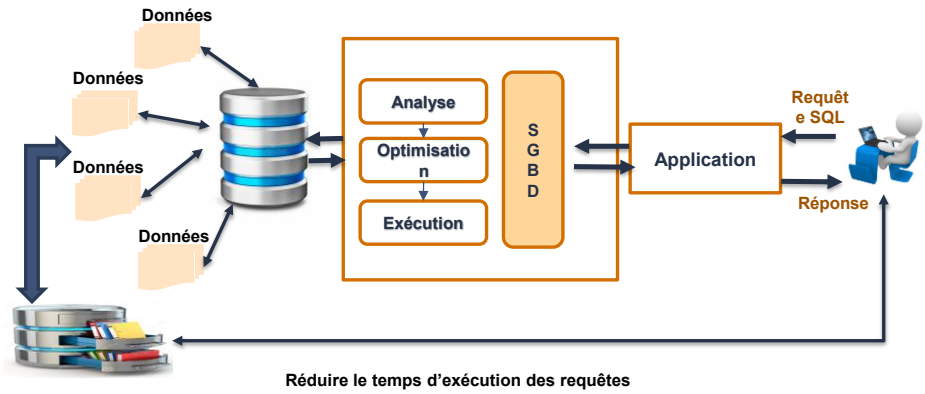
Table Article		
IdLigne	NoArticle	catégorie
A1	10	Santé
A2	20	Santé
A3	40	Santé
A4	50	Sport
A5	60	sport

Table Ventes				
IdLigne (ROWID)	NoClient	NoArticle	dateVente	Montant
V1	1	10	12-03-2015	100
V2	2	20	14-03-2017	200
V3	3	10	12-04-2020	500
V4	1	10	12-03-2023	150
V5	1	60	12-6-2023	400

Table Client		
idLigne (ROWID)	NoClient	nomVille
C1	1	Alger
C2	2	Oran
C3	3	Alger
C4	4	Blida

Index bitmap de jointure

Problématique



Problématique

- Un des objectifs des bases de données est de restituer l'information dans des délais acceptables.
- D'où trouver la meilleure façon d'accéder et de traiter les données afin de répondre le plus rapidement possible à la requête: ceci passe par l'évaluation de la requête :

- L'exécution d'une requête nécessite:
  - Le transfert de données de la mémoire stable MS (disques) vers la mémoire centrale MC
  - Le traitement en MC
  - Écriture si MAJ en MS

Problématique

- L'optimisation consiste à réduire au maximum le volume de ces transferts
- Pour cela, le SGBD dispose d'un module d'évaluation de questions qui permet de choisir la meilleure stratégie possible d'exécution d'une expression relationnelle.
- Plusieurs structures d'optimisation existent:
  - Les structure d'indexation
    - Btree
    - Bitmap
    - Bitmap de jointure
  - La fragmentation
  - Les vues matérialisées
  -

Les techniques d'indexation

- Les index sont des fichiers qui contiennent les paires [attribut(s) d'accès → numéro page]
- Structure permettant d'associer à une clé d'un tuple l'adresse physique correspondante
- Un index primaire est construit sur la clé primaire d'une relation
- Un index secondaire est construit sur un attribut quelconque d'une relation

Les choix d'oracle

- **Par défaut l'index est un arbre B**
  - Dès qu'on utilise une commande PRIMARY KEY, Oracle crée un arbre B sur la clé primaire
  - Arbre est stocké dans un segment d'index

  - L'index B arbre est généralement définie sur les attribut ayant plusieurs valeurs distinctes
  - Cependant, dans la clause where d'une requête SQL on utilise d'autres attributs

Principe de l'index Bitmap

Soit un attribut A, ayant prenant n valeurs possibles (V1, …, Vn)

- **Création d'un index bitmap sur l'attribut A:**
  - On crée n **tableaux** de **n bits**, un pour chaque ***valeur vi***
  - Le tableau contient un bit pour chaque tuple t
  - Le bit d'un tuple t est à 1 si t.A=vi, à 0 sinon

- **Création d'un index bitmap**

```
CREATE  BITMAP INDEX nom_index
 ON  Nom_TABLE.Nom_attribut
```

Utilisation de l'index Bitmap

- **Exemple: Select * From** Enseignant **where** Grade ='Prof'
- **Syntaxe de création de l'index**

CREATE BITMAP INDEX Grade_ib_index
ON Enseignant.Grade

- **Important:**
  - Lorsque le system (OS) stocke les données dans la mémoire secondaire, il envoi à l'ASGBD l'adresse physique de l'enregistrement
  - L'SGBD stocke cette adresse dans la colonne RowID
  - Il s'agit numéro convertible en adresse physique

Enseignant			
ROWID(RID)	ID_ENS	Nom	Grade
00076:000:0123	1	Kadi	Prof
00123:045:8954	2	Melyani	MCA
03458:020:6543	3	Riahi	Prof
12065:213:1003	4	Slimani	MCB
16065:654:1234	5	Fertoul	MCB
23876:020:344	6	Berzeg	MAA

Utilisation de l'index Bitmap

- **Exemple:**

Enseignant			
ROWID(RID)	ID_ENS	Nom	Grade
00076:000:0123	1	Kadi	Prof
00123:045:8954	2	Melyani	MCA
03458:020:6543	3	Riahi	Prof
12065:213:1003	4	Slimani	MCB
16065:654:1234	5	Fertoul	MCB
23876:020:344	6	Berzeg	MAA

Index Bitmap				
ROWID(RID)	Prof	MCA	MCB	MaA
00076:000:0123	1	0	0	0
00123:045:8954	0	1	0	0
03458:020:6543	1	0	0	0
12065:213:1003	0	0	1	0
16065:654:1234	0	0	1	0
23876:020:344	0	1	0	0

Utilisation de l'index Bitmap

- Soit la requête : **Select * From** Enseignant **where** Grade ='Prof'
  - **Exécution de la requête sans index :** le système charge toute la table Enseignant
  - **Exécution de la requête avec l'index:** le système
    - Charge l'index en RAM
    - Comme la restriction est définie sur grade =Prof, il se positionne sur la colonne Prof
    - seules les RowID des TupleS ayant la valeur 1 dans la colonne prof seront récupérés
    - Une fois récupérer les adresses physiques, il les envoient à l'OS pour récupérer les enregistrements des ces RowID

Exemple

- Table Enseignant: un index bitmap est définit sur l'attribut Grade

Table enseignants

ROWID(RID)	ID_ENS	Nom	Grade
00076:000:0123	1	Kadi	Prof
00123:045:8954	2	Melyani	MCA
03458:020:6543	3	Riahi	Prof
12065:213:1003	4	Slimani	MCB
16065:654:1234	5	Fertoul	MCB
23876:020:344	6	Berzeg	MAA

Index binaire sur l'attribut grade

ROWID(RID)	Prof	MCA	MCB	MAA
00076:000:0123	1	0	0	0
00123:045:8954	0	1	0	0
03458:020:6543	1	0	0	0
12065:213:1003	0	0	1	0
16065:654:1234	0	0	1	0
23876:020:344	0	0	0	1

La taille de l'index Bitmap

- Taille de l'index définit sur un attribut A de la table T

|index | = (|DIST (A) | + RID) * || T ||

  - DIST (A): représente la cardinalité de l'attribut A
  - ||T||: le nombre de ligne de la table T
  - RID: la taille du RID

- La taille de l'index devient très élevée dans le cas ou l'attribut a plusieurs valeurs distincts ou bien lorsque la taille de la table est très importante
  - **Le SGBD utilise des technique de compression pour réduire la taille de l'index**

Utilisation de l'index Bitmap

Exemple: soit la requête R:

SELECT *
From Enseignant
WHERE Grade ='MCB' **OR** grade ='MCA'

SELECT *
From Enseignant
WHERE Grade IN ('MCB' , 'MCA')

ROWID(RID)	Prof	MCA	MCB	MCA
00076:000:0123	1	0	0	0
00123:045:8954	0	1	0	0
03458:020:6543	1	0	0	0
12065:213:1003	0	0	1	0
16065:654:1234	0	0	1	0
23876:020:344	0	1	0	0

MCB	v	MCA	=	MCAOR MCB
0		0		0
0		1		1
0		0		0
1		0		1
1		0		1
0		0		0

ROWID(RID)	ID_ENS	Nom	Grade
00076:000:0123	1	Kadi	Prof
00123:045:8954	2	Melyani	MCA
03458:020:6543	3	Riahi	Prof
12065:213:1003	4	Slimani	MCB
16065:654:1234	5	Fertoul	MCB
23876:020:344	6	Berzeg	MAA

Index BITMAP

- Ce type d'index permet d'optimiser les requête de sélection
  - La sélection est enfaite précalculer lors de la création de l'index

Utilisation de l'index Bitmap

Exemple: soit la requête R:
SELECT count (*)
From Enseignant
WHERE Grade ='MCB' **OR** grade ='MCA'

SELECT count (*)
From Enseignant
WHERE Grade IN ('MCB' , 'MCA')

ROWID(RID)	Prof	MCA	MCB	MCA
00076:000:0123	1	0	0	0
00123:045:8954	0	1	0	0
03458:020:6543	1	0	0	0
12065:213:1003	0	0	1	0
16065:654:1234	0	0	1	0
23876:020:344	0	1	0	0

MCB	v	MCA	=	MCA
0		0		0
0		1		1
0		0		0
1		0		1
1		0		1
0		0		0

Index BITMAP

- Les index bitmap est intéressant pour les requêtes de types select count(*)
  - Le système pourra dans ce cas répondre à la requête en chargeons uniquement l'index

Plan d'exécution

- **Création de l'index**

```
CREATE BITMAP INDEX members_gender_i
ON members(gender);
```

- **Plan d'execution**

```
EXPLAIN PLAN FOR
    SELECT * FROM members WHERE gender = 'F';

SELECT PLAN_TABLE_OUTPUT
FROM TABLE(DBMS_XPLAN.DISPLAY());
```

Plan d'exécution d'une requête

EXPLAIN PLAN FOR **SELECT * FROM** members **WHERE** gender = 'F';

PLAN_TABLE_OUTPUT

```
Plan hash value: 1733317867

---------------------------------------------------------------------------------------------
| Id  | Operation                          | Name             | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                   |                  |   509 |  128K|     3   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID BATCHED| MEMBERS          |   509 |  128K|     3   (0)| 00:00:01 |
|   2 |   BITMAP CONVERSION TO ROWIDS      |                  |       |       |            |          |
|*  3 |    BITMAP INDEX SINGLE VALUE       | MEMBERS_GENDER_I |       |       |            |          |
---------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - access("GENDER"='M')

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
```

3: Le système a utilisé l'index
2: Le système a convertit ce qu'il a trouvé comme 1 en ROID
1: accèder à la table Members en utilisant les RID
0: Répondre à la requêtes SELECT *

Exercice de TD

Exercice 1

Soit une base de données composée de trois tables Client, Produit et Ventes. La table Ventes contient les clés étrangères des deux autres table en plus de la quantité vendu et le prix unitaire de la vente.

Ventes			
CID	OID	Qte	Prix ventes
616	106	5	25
616	104	10	50
515	104	5	10
414	105	2	14
212	106	4	14
111	101	5	20
212	101	9	100
313	102	3	200
414	102	22	102
515	102	2	100
515	103	4	17
212	103	6	45
111	105	89	44
212	104	8	40
515	104	7	20
616	104	5	20
212	105	2	10
212	105	2	10
212	106	4	18
313	105	4	19
313	106	6	15

Client				
CID	Nom	Ville	Age	Genre
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
313	Karima	Tipaza	76	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F

Produit		
PID	Nom	Type
106	Sonoflore	Alger
105	Clarins	Blida
104	Webcam	Tipaza
103	Barbie	Tipaza
102	Engrais	Alger
101	Slimform	Alger

Exercice 1

Soient les requêtes suivantes :

R1 : **SELECT** Nom **FROM** Client **WHERE** Ville = ‘Alger ‘ ;

R2 : **SELECT** * **FROM** Client **WHERE** genre =’M’

R3 : **SELECT** * **FROM** Client **WHERE** Genre = ‘M’ or Ville = ‘Alger’

1. **Donner le résultat d’exécution de chaque requêtes**

a) La requête R1

Client				
CID	Nom	Ville	Age	Genre
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
313	Karima	Tipaza	76	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F

→

Nom
Ghilles
Ali
Yasmine

Exercice 1

Soient les requêtes suivantes :

R1 : **SELECT** Nom **FROM** Client **WHERE** Ville = 'Alger ' ;

R2 : **SELECT** * **FROM** Client **WHERE** genre ='M'

R3 : **SELECT** * **FROM** Client **WHERE** Genre = 'M' or Ville = 'Alger'

1. **Donner le résultat d'exécution de chaque requêtes**

a) La requête R2

Client				
CID	**Nom**	**Ville**	**Age**	**Genre**
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
313	Karima	Tipaza	76	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F

Client				
CID	**Nom**	**Ville**	**Age**	**Genre**
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
212	Ali	Alger	88	M

Exercice 1

2. **Donner la requête permettant de créer un index bitmap pour optimiser la requête R1**

R1 : **SELECT** Nom **FROM** Client **WHERE** Ville = 'Alger ' ;

Il s'agit de créer un index Bitmap sur l'attribut Ville de la table Client

Syntaxe:

CREATE BITMAP INDEX Index_ville
ON Client(Ville);

Exercice 1

3. **Donner la requête permettant de créer un index Bitmap pour optimiser la requête R2**

R2 : **SELECT** * **FROM** Client **WHERE** genre ='M'

- Il s'agit de créer un index Bitmap sur l'attribut Ville de la table Client
- **Syntaxe:**

CREATE BITMAP INDEX Index_genre
ON Client(genre);

Exercice 1

4. **Dessiner les deux index et montrer l'utilisation de cet index afin de trouver le même résultat des deux requêtes**

CREATE BITMAP INDEX Index_ville **ON** Client(Ville);

Client				
CID	Nom	Ville	Age	Genre
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
313	Karima	Tipaza	76	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F

Index_ville			
RID	Alger	Blida	Tipaza
	1	0	0
	0	1	0
	0	0	1
	0	0	1
	1	0	0
	1	0	0

La taille de l'index: (3+ RID) * 6 = ….. Bits

Exercice 1

4. **Dessiner les deux index et montrer l'utilisation de cet index afin de trouver le même résultat des deux requêtes**

CREATE BITMAP INDEX Index_genre **ON** Client(genre);

Client				
CID	Nom	Ville	Age	Genre
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
313	Karima	Tipaza	76	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F

Index_genre		
Rid	M	F
	1	0
	1	0
	0	1
	0	1
	1	0
	0	1

La taille de l'index: (2+ RID) * 6 = ….Bits

- Charger l'index en RAM

Exercice 1

5. **Comment peut-on- exploiter les deux index pour répondre à la requête R3**

- Charger les deux indexes
- Effectuer un OR logique entre la colonne Alger et la colonne M
- Récupérer les RID
- Chercher les CID dans la table Client qui correspondent aux lignes dont résultat = 1

Index_ville			
CID	Alger	Blida	Tipaza
616	1	0	0
515	0	1	0
414	0	0	1
313	0	0	1
212	1	0	0
111	1	0	0

OR

Index_genre		
CID	M	F
616	1	0
515	1	0
414	0	1
313	0	1
212	1	0
111	0	1

=

Alger Or M
1
1
0
0
1
1

Client				
CID	Nom	Ville	Age	Genre
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
313	Karima	Tipaza	76	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F

L’index de jointure BITMAP

Index de jointure

- Pré-calcule une opération de jointure
- Utilisée avec les tables liées avec des clés étrangères avec plusieurs tables
- Maintient des relation entre une clés étrangères avec les clés primaires
- Exemple

Index de jointure

- Exemple : Trouver le montant de ventes des article de type Electroménager

Index de jointure		
Id-ligneVente	**Id-ligneClient**	**Id-ligneArticle**
V1	**C1**	**A1**
V2	C2	A2
V3	C3	A1
V4	C1	A1

Table client

Id-ligne	CodeClient	Ville
C1	**1**	**Blida**
C2	2	Boumerdes
C3	3	Blida
C4	4	Alger

Table ventes

N°ligne	CodeClient	Numarticle	DateVente	Montant
V1	**1**	**10**	**18-12-2021**	**1000**
V2	2	20	14-03-2017	2000
V3	3	10	12-03-2017	5000
V4	1	10	1-04-2016	3000

Table article

Id-ligne	Numarticle	catégorie
A1	**10**	**électroménager**
A2	20	Electroménager
A3	30	Electroménager
A4	40	Mode

- Charger la table Article
- Récupérer le résultat de la sélection : A1, A2 , A3
- Charger l'index , et récupérer les RID qui correspondent à ces valeurs
- Le système envoi ces RID à l'OS, pour récupérer les montant
- Puis calculer la somme

L'index de jointure Bitmap

- Il permet d'optimiser les jointures
- Il est utilisable si les jointures se fond sur les clés étrangères

- **Création de l'index Bitmap de jointure**

CREATE BITMAP INDEX nom_index
ON Table2(table1.att3)
FROM table2, table1
Where a completer

Sachant que att2 est une clé étrangère → att

Exemple d'un index Bitmap de Jointure

- **Exemple**

Client(CodeClient, Ville)
Article(CodeArticle, catégorie)
Vente (CodeClient, CodeArticle, date, Montant)

- **Exemple de création de l'index**

CREATE BITMAP INDEX ventes_bji
ON Ventes (Client.Ville)
FROM Ventes, Client
WHERE Ventes.CodeClient= Client.codeClient;

Index Bitmap de jointure

RID	Id-ligne	CodeClient	Ville
	C1	1	Blida
	C2	2	Boumerdes
	C3	3	Blida
	C4	4	Alger

RID	N°ligne	CodeClient	Numarticle	DateVente	Montant
	V1	1	10	18-12-2021	1000
	V2	2	20	14-03-2017	2000
	V3	3	10	12-03-2017	5000
	V4	1	10	1-04-2016	3000

SELECT count(*)
FROM Ventes V, Client C
WHERE V;CID= C.CID
AND C.ville ='Blida'

CREATE BITMAP INDEX ventes_bji
ON Ventes (Client.Ville)
FROM Ventes, Client
WHERE Ventes.CodeClient= Client.codeClie

Index			
RID	Alger	Blida	Boumerdes
	0	1	0
	0	0	1
	0	1	0
	0	1	0

Select count (*) From ventes, v Client c Where c.Ville = 'Blida' and c.CodeClient= v.CodeClient

Id-ligne	CodeClient	Ville
C1	1	Blida
C2	2	Boumerdes
C3	3	Blida
C4	4	Alger

Id-ligneVente	Blida	Boumerdec	Alger
V1	1	0	0
V2	0	1	0
V3	1	0	0
V4	0	0	1

N°ligne	CodeClient	Numarticle	DateVente	Montant
V1	1	10	18-12-2021	1000
V2	2	20	14-03-2017	2000
V3	3	10	12-03-2017	5000
V4	1	10	1-04-2016	3000

Lire dans le bitmap la colonne P
Compter le nombre de 1
→ Pas de lecture dans la table ventes

- Deux opérations
  - Lire dans le bitmap la colonne P
  - Compter le nombre de 1

→ pas de lecture de la table Ventes

Exercice de TD

Exercice 1

Soit la requête **R4** : **SELECT** Count (*)
FROM Client, Ventes
WHERE Client.CID= VENTES.CID **AND** CLIENT.VILLE ='Alger'

6. **Donner le résultat de la requête**

Client				
CID	**Nom**	**Ville**	**Age**	**Genre**
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
313	Karima	Tipaza	76	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F

Ventes			
CID	**OID**	**Qte**	**Prix ventes**
616	106	5	25
616	104	10	50
545	104	5	10
414	105	2	14
212	106	4	14
111	101	5	20
212	101	9	100
313	102	3	200
414	102	22	102
515	102	2	100
515	103	4	17
212	103	6	45
111	105	89	44
212	104	8	40
515	104	7	20
616	104	5	20
212	105	2	10
212	105	2	10
212	106	4	18
313	105	4	19
313	106	6	15

→ **12 Enregistrements**

Exercice 1

7. Donner la requête permettant de créer un index de jointure bitmap pour optimiser la requête R4

CREATE BITMAP INDEX Index_vente_ville_ijb
ON VENTES(C.Ville)
FROM Ventes V, Client C
WHERE V.CID=C.CID

8. Dessiner l'index résultat et montrer l'utilisation de cet index afin de trouver le même résultat des deux requêtes

	Ventes			
RID	CID	OID	Qte	Prix ventes
	616	106	5	25
	616	104	10	50
	545	104	5	10
	414	105	2	14
	212	106	4	14
	111	101	5	20
	212	101	9	100
	313	102	3	200
	414	102	22	102
	515	102	2	100
	515	103	4	17
	212	103	6	45
	111	105	89	44
	212	104	8	40
	515	104	7	20
	616	104	5	20
	212	105	2	10
	212	105	2	10
	212	106	4	18
	313	105	4	19
	313	106	6	15

	Client				
RID	CID	Nom	Ville	Age	Genre
	616	Ghilles	Alger	26	M
	515	Mohamed	Blida	15	M
	414	Salma	Tipaza	55	F
	313	Karima	Tipaza	76	F
	212	Ali	Alger	88	M
	111	Yasmine	Alger	30	F

Comment l'index est crée?

	Ventes			
RID	CID	OID	Qte	Prix ventes
	616	106	5	25
	616	104	10	50
	515	104	5	10
	414	105	2	14
	212	106	4	14
	111	101	5	20
	212	101	9	100
	313	102	3	200
	414	102	22	102
	515	102	2	100
	515	103	4	17
	212	103	6	45
	111	105	89	44
	212	104	8	40
	515	104	7	20
	616	104	5	20
	212	105	2	10
	212	105	2	10
	212	106	4	18
	313	105	4	19
	313	106	6	15

CID	Nom	Ville	Age	Genre
616	Ghilles	Alger	26	M
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F
212	Ali	Alger	88	M
313	Karima	Tipaza	76	F
414	Salma	Tipaza	55	F
515	Mohamed	Blida	15	M
515	Mohamed	Blida	15	M
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F
212	Ali	Alger	88	M
515	Mohamed	Blida	15	M
616	Ghilles	Alger	26	M
212	Ali	Alger	88	M
212	Ali	Alger	88	M
212	Ali	Alger	88	M
313	Karima	Tipaza	76	F
313	Karima	Tipaza	76	F

	Client				
RID	CID	Nom	Ville	Age	Genre
	616	Ghilles	Alger	26	M
	515	Mohamed	Blida	15	M
	414	Salma	Tipaza	55	F
	313	Karima	Tipaza	76	F
	212	Ali	Alger	88	M
	111	Yasmine	Alger	30	F

Pour chaque ligne de la table ventes est Concaténée la ligne correspondante De la table Client

Après le système créé l'index sur le résultat

	Ventes			
RID	CID	OID	Qte	Prix ventes
1	616	106	5	25
2	616	104	10	50
3	515	104	5	10
4	414	105	2	14
5	212	106	4	14
6	111	101	5	20
7	212	101	9	100
8	313	102	3	200
9	414	102	22	102
10	515	102	2	100
11	515	103	4	17
12	212	103	6	45
13	111	105	89	44
14	212	104	8	40
15	515	104	7	20
16	616	104	5	20
17	212	105	2	10
18	212	105	2	10
19	212	106	4	18
20	313	105	4	19
21	313	106	6	15

	Client			
CID	Nom	Ville	Age	Genre
616	Ghilles	Alger	26	M
616	Ghilles	Alger	26	M
515	Mohamed	Blida	15	M
414	Salma	Tipaza	55	F
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F
212	Ali	Alger	88	M
313	Karima	Tipaza	76	F
414	Salma	Tipaza	55	F
515	Mohamed	Blida	15	M
515	Mohamed	Blida	15	M
212	Ali	Alger	88	M
111	Yasmine	Alger	30	F
212	Ali	Alger	88	M
515	Mohamed	Blida	15	M
616	Ghilles	Alger	26	M
212	Ali	Alger	88	M
212	Ali	Alger	88	M
212	Ali	Alger	88	M
313	Karima	Tipaza	76	F
313	Karima	Tipaza	76	F

	Index		
RID	Alger	Blida	Tipaza
1	1	0	0
2	1	0	0
3	0	1	0
4	0	0	1
5	1	0	0
6	1	0	0
7	1	0	0
8	0	0	1
9	0	0	1
10	0	1	0
11	0	1	0
12	1	0	0
13	1	0	0
14	1	0	0
15	0	1	0
16	1	0	0
17	1	0	0
18	1	0	0
19	1	0	0
20	0	0	1
21	0	0	1

Comment l'index est utilisé pour répondre à la requête R4

R4 : **SELECT** Count (*)
FROM Client, Ventes
WHERE Client.CID= VENTES.CID **AND** CLIENT.VILLE ='Alger'

1. Le système charge l'index
2. Il se positionne sur la colonne Alger
3. Il compte le nombre de 1
4. La valeur trouvé est 12, →optimiser une jointure et sélection sans charger les tables

La fragmentation

Fragmentation

- La fragmentation est le processus de **décomposition** d'une base de donnée logique en un ensemble de "sous" bases de données.
- Cette décomposition doit être sans perte d'information.
- La fragmentation peut être coûteuse s'il existe des applications qui possèdent des besoins opposés.
- Le processus de fragmentation peut se faire sous différentes stratégies: horizontale, verticale ou hybride.
- Tout stratégie de fragmentation doit satisfaire les trois règles de correctitude : **complétude**, **reconstruction** et **disjonction**.

Fragmentation

Règle de correction

- **Règle de completion:** pour toute donnée d'une relation R, il existe un fragment Ri de la relation R qui possède cette donnée → pas de perte d'information
- **Règle de reconstruction:** il est toujours possible de reconstruire la relation R par un opérateur algébre relationelle
- **Règle de disjoncteur:** si une donnée apparait dans un fragment alors elle ne peut apparaitre dans aucun autre fragment sauf on autorise la redondance de l'attribut jointure pour la fragmentation verticale qui fait le lien entre les fragment

Fragmentation

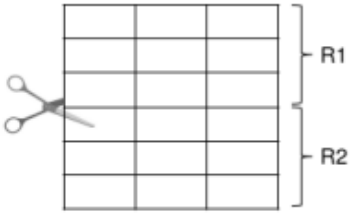
Il existe trois type de fragmentation

- **Fragmentation horizontale :** la table est fragmentée par rapport à ses instances en un ensemble de ligne
- **Fragmentation verticale:** la table est fragmentée selon ses attribut en un ensemble de colonne
- **Fragmentation mixte:** la table est fagmentée horizontalement et verticalement

1. Fragmentation horizontale FH

Définition

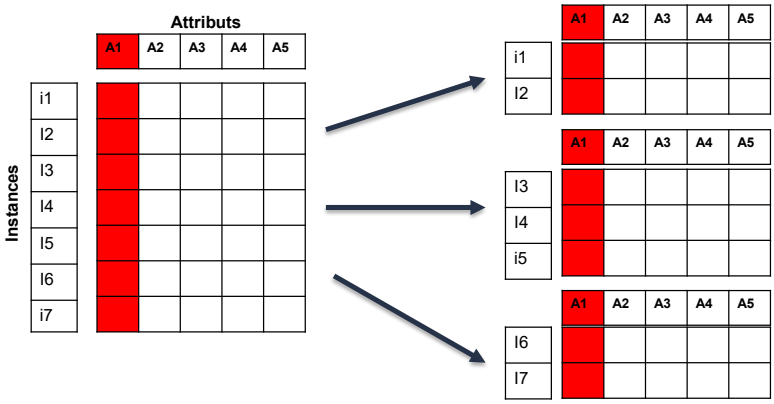
- Ce type de fragmentation permet de découper une relation en sous relations contenants des sous-ensembles des tuples de la relation mère.



- L'opérateur de fragmentation est **la restriction**
- L'opérateur de reconstruction de R est **l'union**

  R= R1 union R2 … union Rn

1. Fragmentation horizontale FH



1.Fragmentation horizontale FH

Définition

- La fragmentation horizontale est basée sur **l'opération de sélection** qui utilise **un prédicat (condition).**
- La reconstitution de la relation se fait par l'union des fragments.
- Nous avons besoin de définir deux notions: **prédicat simple**, et **prédicat minterm.**

Prédicat simple:

- Soit une relation R(A1 , A2 , ..., An) , tel que Ai est un attribut de R défini sur un domaine Di. Un prédicat simple pj est défini sur R comme suit: **pj : Ai θ val**

Tel que θ ∈ { = , <, >, ≠ , ≥ , ≤} et val ∈ Di.

On note Pr l'ensemble de tous les prédicats simples définis sur R.

1. Fragmentation horizontale FH

Exemple de prédicat simple:

- Soit une entreprise qui a des annexes dans plusieurs villes: **Alger**, **Sétif**, **Blida**. Le gérant veut maintenir une base de données des employés et les projets sur lesquels ils travaillent.

- Le gérant utilise 4 tables comme suit:
  - **Employés** (idEmp, nomEmp, poste);
  - **Projetj** (idPrj, nomPrj, designation, budget, ville);
  - **Salaire** (poste, salaire);
  - **Affectation** (idEmp, idPrj, resp, dur) // Affectation d'un employé à un projet avec telle responsabilité et telle durée.

- Soit la relation Projet, on peut définir quelques prédicats simples:
  - nomPrj = "Construction "
  - budget ≤ 100000

1. Fragmentation horizontale FH

Les types de fragmentation horizontale

- Il existe deux types de fragmentation horizontale:
  - primaire
  - et dérivée.

1. Fragmentation horizontale FH

Fragmentation horizontale primaire

- FHP est la fragmentation horizontale qui est basée sur l'opération de sélection qui utilise un prédicat minterm:

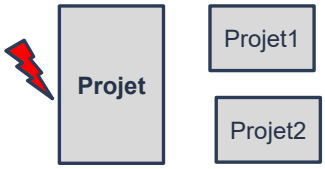
$$R_i = \sigma\, m_i\, (R)$$

- Tel que :
  - R_i est un fragment de la relation R.
  - m_i est un minterm.
  - σ est l'opération de sélection

1. Fragmentation horizontale FH

Exemple 1: Fragmentation horizontale primaire

- Fragmentation de la relation Prj de l'exemple précédent en fragmentation horizontale peut être faite comme suit:
  - Projet1 = $\sigma_{budget \leq 100000}$ (Prj)
  - Projet2 = $\sigma_{budget > 100000}$ (Prj)

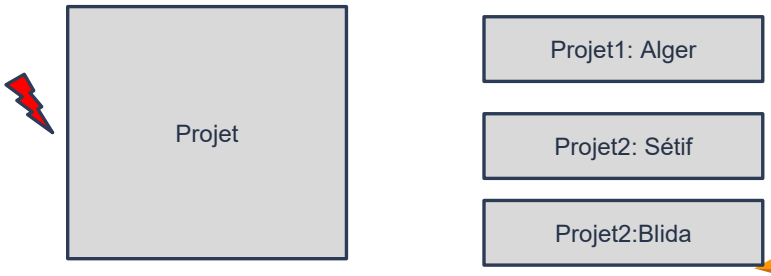


- Cet exemple présente un des problèmes de la fragmentation horizontale. Si le domaine des attributs est continu et infini il sera difficile de définir le nombre maximal de fragmentations possibles. On ne peut pas partager le domaine en un nombre fini de sous-domaines

1. Fragmentation horizontale FH

Exemple 2: Fragmentation horizontale primaire

- On peut fragmenter maintenant la relation Projet en se basant sur la localisation comme suit:
  - Projet 1 = σ ville= "Alger" (Projet)
  - Projet 2 = σ ville= "Setif" (Projet)
  - Projet 3 = σ ville= "Blida" (Projet)
- Dans ce cas, le nombre maximal de fragmentations possibles est fixe



1. Fragmentation horizontale FH

- Pour éviter les problèmes de fragmentation horizontale, nous avons besoin de bien **choisir les prédicats** simples qui seront à la base des prédicats minterm permettant de fragmenter une relation.

- D'une manière générale, cet ensemble doit **contenir seulement** les prédicats avec des **attributs** et **conditions** utilisés par les applications.

1. Fragmentation horizontale FH

Fragmentation horizontale dérivée

- La FHD d'une relation(R) est basée sur des fragments horizontaux d'une autre relation(S) à condition qu'il y est un lien entre R et S.

- Les nouveaux fragments sont définis par semi-jointure.

- **Rappel:** La semi-jointure entredeux relations R et S ($R \ltimes S$) permet de garder les tuples de R qui ont relation avec des tuples de S.

1. Fragmentation horizontale FH

Exemple: Fragmentation horizontale dérivée

- Dans l'exemple précédent, nous voulons grouper les employés de la relation **Employés (idEmp, nomEmp, poste)** en 2 groupes selon leurs salaires. Un groupe pour ceux qui ont un salaire **inférieur à 30000**, et les autres(**≥ 30000**) dans le deuxième groupe.
- On fragmente d'abord la relation Sal(poste, salaire) en deux fragments comme suit:
  - **Salaire1** =σ salaire< 30000 (Salaire)
  - **Salaire2** =σ salaire≥ 30000 (Salaire)
- Les deux fragments contenant les deux groupes de salariés sont définis ensuite par <u>semi-jointure</u> sur la colonne "poste " pour définir l'ensemble final des fragments comme suit:
  - **Employés1** =Employés $\ltimes_{poste}$ **Salaire1**
  - **Employés2** = Employés $\ltimes_{poste}$ **Salaire2**
  - **Employés1 = projection(les attibu employes)** (Employés Join $_{poste}$ **Salaire1)**

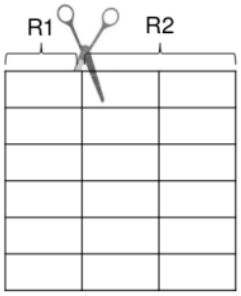
Poste	Salaire
Comptable	20000
Ingenieur	65555
admin	25000
technicien	45555

Poste	Salaire
Comptable	20000
admin	25000

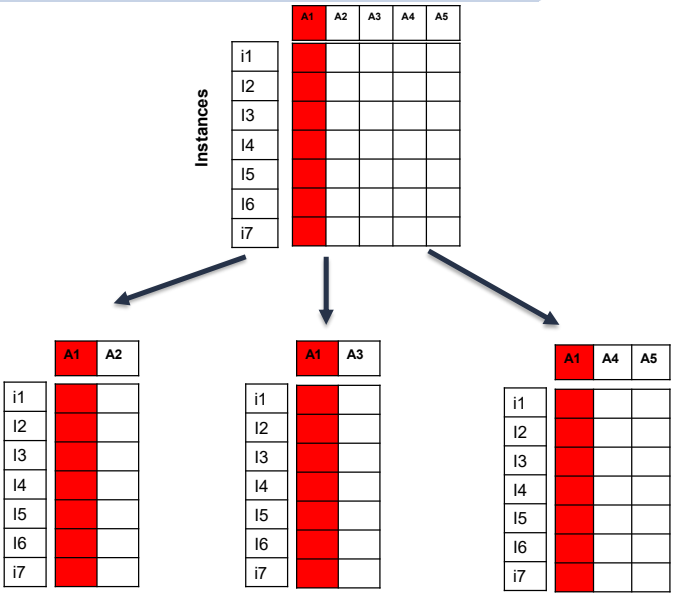
Poste	Salaire
Ingenieur	65555
technicien	45555

2. Fragmentation verticale

- **La fragmentation verticale** permet de découper une relation R en plusieurs relations. Chaque fragment contient un sous-ensemble d'attributs ainsi que la clé primaire de R.



2. Fragmentation verticale



2. Fragmentation verticale

- Une relation R *fragmentée verticalement* est décomposée en plusieurs sous ensemble R1, R2, … ,Rn telque: chaque

  Ri= project (R / Ri)

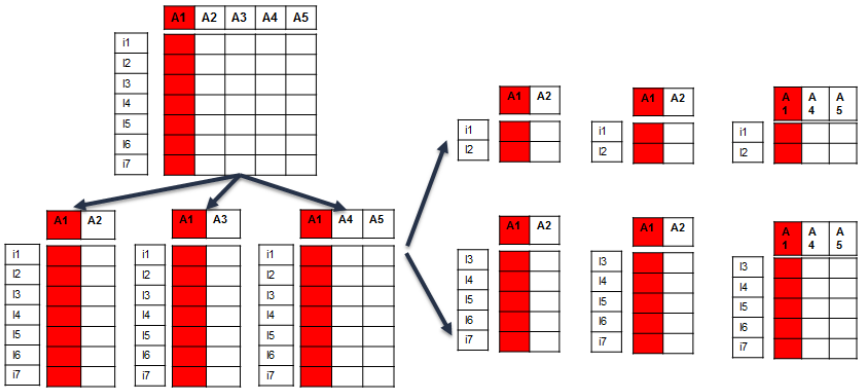
  - L'opérateur de fragmentation est **la projection**

  - L'opérateur de reconstruction de R est **la jointure**

    R= R1 join R2 … join Rn

3. Fragmentation mixte

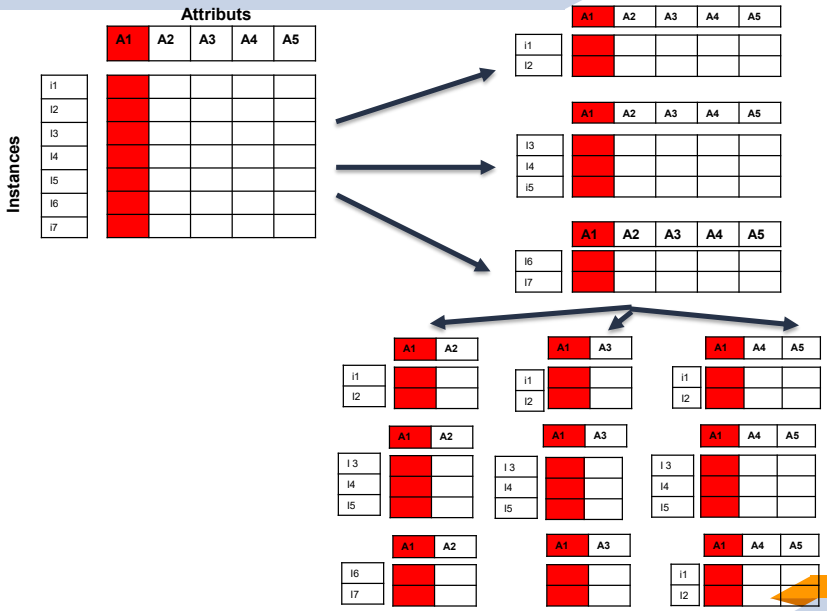
- La Fragmentation Mixte(FM) (ou hybride) est une combinaison des deux fragmentations précédentes (FH etFV).
  - **Verticale suivie d'une horizontale**



3. Fragmentation mixte

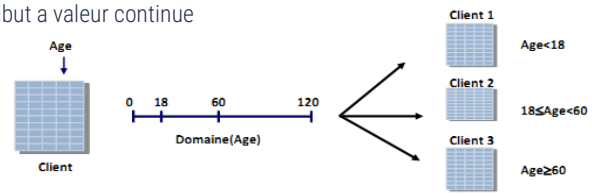
- La Fragmentation Mixte(FM) (ou hybride) est une combinaison des deux fragmentations précédentes (FH etFV).
  - Horizontale suivie d'une verticale

3. Fragmentation mixte



Fragmentation par intervalle (domaine) « Range »

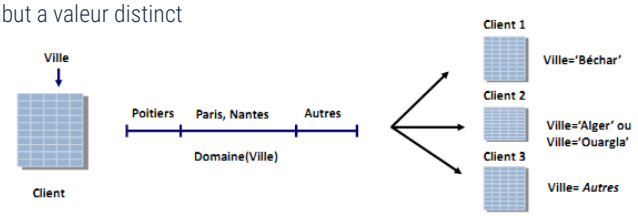
- La fragmentation se fait sur un attribut a valeur continue




```
CREATE TABLE Client
(  Client_id NUMBER(5),
   Nom Varchar2(20),  Ville Varchar2(20),
   Age Number(3),
   Genre Varchar2(1))
PARTITION BY RANGE (Age)  (
 PARTITION Client_Moins_18 VALUES LESS THAN (18) TABLESPACE  TBSMoins27,
PARTITION Client_18_59 VALUES LESS THAN (60) TABLESPACE TBS27-59,
PARTITION Client_60_Et_Plus VALUES LESS THAN (MAXVALUES) TABLESPACE TBSPlus60
);
```

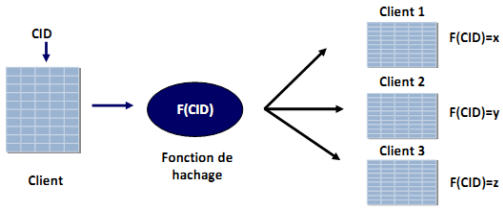
Oracle: fragmentation par Liste

- La fragmentation se fait sur un attribut a valeur distinct




```
CREATE TABLE Client
( Client_id NUMBER(5), Nom Varchar2(20), Ville Varchar2(20), Age Number(3),
  Genre Varchar2(1),
  PARTITION BY List (Ville) (
     PARTITION Client_Béchar VALUES ('Béchar') TABLESPACE TBSBECHAR,
     PARTITION Client_Alg_Oua VALUES ('Alger','Ouargla') TABLESPACE TBSALGOUA,
     PARTITION Client_Autres VALUES (DEFAULT) TABLESPACE TBSAUTRES
);
```


Oracle: fragmentation par hash



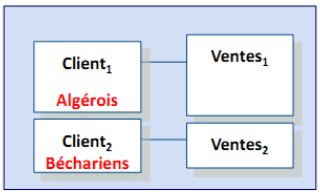

```
CREATE TABLE Client
(Client_id NUMBER(5),
Nom Varchar2(20),
Ville Varchar2(20),
Age Number(3),
Genre Varchar2(1),
PARTITION BY Hash (CID)
(
   PARTITIONS 3   STORE IN (TBS1, TBS2, TBS3)
);
```

Fragmentation composite

```
CREATE TABLE Client
(Client_id NUMBER(5),
Nom Varchar2(20),  Ville Varchar2(20),  Age Number(3),
Genre Varchar2(1))
PARTITION BY RANGE (Age),
SUBPARTITION BY LIST (Genre)
SUBPARTITION TEMPLATE
(
   SUBPARTITION Client1 VALUES ('M') TABLESPACE TBSMasculin ,
   SUBPARTITION Client2 VALUES ('F') TABLESPACE TBSFéminin )
(
PARTITION Client_Moins_18 VALUES LESS THAN (18),
PARTITION Client_18_59 VALUES LESS THAN (60),
PARTITION Client_60_Et_Plus VALUES LESS THAN (MAXVALUES)
);
```


Fragmentation drivée par mode de référence

- Fragmenter une table selon le schéma de fragmentation d'une autre table en utilisant le lien par clé étrangère




```
CREATE TABLE Ventes (  Client_id NUMBER(5),  Time_id NUMBER(5),
Montant NUMBER(20),
CONSTRAINT order_items_fk  FOREIGN KEY (Client_id) REFERENCES
Client (Client_id)
)
PARTITION BY REFERENCE (order_items_fk);
```

Allocation des deonnées

- Allocation centralisée
  - Une seule base de données, un seule SGBD, sur un site des utilisateurs répartis sur le réseau. Tous les sites sauf le site centrale doivent passer par le réseau pour tous les accès aux données
  - Coût de communication très élevés fiabilité et disponibilité de données réduits
- Allocation fragmentée ou partitionnée
  - partitionnement de la bases de données en fragments disjoints distribués chacun sur un site
  - Cout de stockage et de communication réduits mais fiabilité et disponibilité réduites

- Réplication complète
  - Une copie de la totalité de la BD sur chaque site
  - Fiabilité disponibilité optimale mais cout de stockage et de communication élevé a cause des mises a jours
- Réplication sélective
  - Combinaison de la centralisation, fragmentation et réplication
  - Tous les avantages de chaque approche toute en évitant les inconvénients

Recherche dans une table avec index

Soit la table **Etudiant** (matricule, nom, prenom, age, section)

Exemple: **SELECT** * **FROM** Etudiant **WHERE** Age = 24;

- Un index sur Age permet d'obtenir un accès plus rapide sur les enregistrements des tuples des employés de 24 ans.

Requêtes conjonctives

SELECT *

FROM Etudiant

WHERE age = 24 **AND** Section = `M1 ';

- Supposons l'existence de deux index sur la relation Etudiant ; un sur **Age** et l'autre sur **Section**.
- L'utilisation du premier index va fournir les enregistrements satisfaisant la 1ère condition qui seront regroupés dans une table temporaire T1. Idem pour le deuxième index ~ T2
- Enfin l'intersection de T1 et T2 donnera la réponse